

MACHINE LEARNING IN PHYSICS

GRAPH NEURAL NETWORKS

HARRISON B. PROSPER

PHY6937 / PHY4936

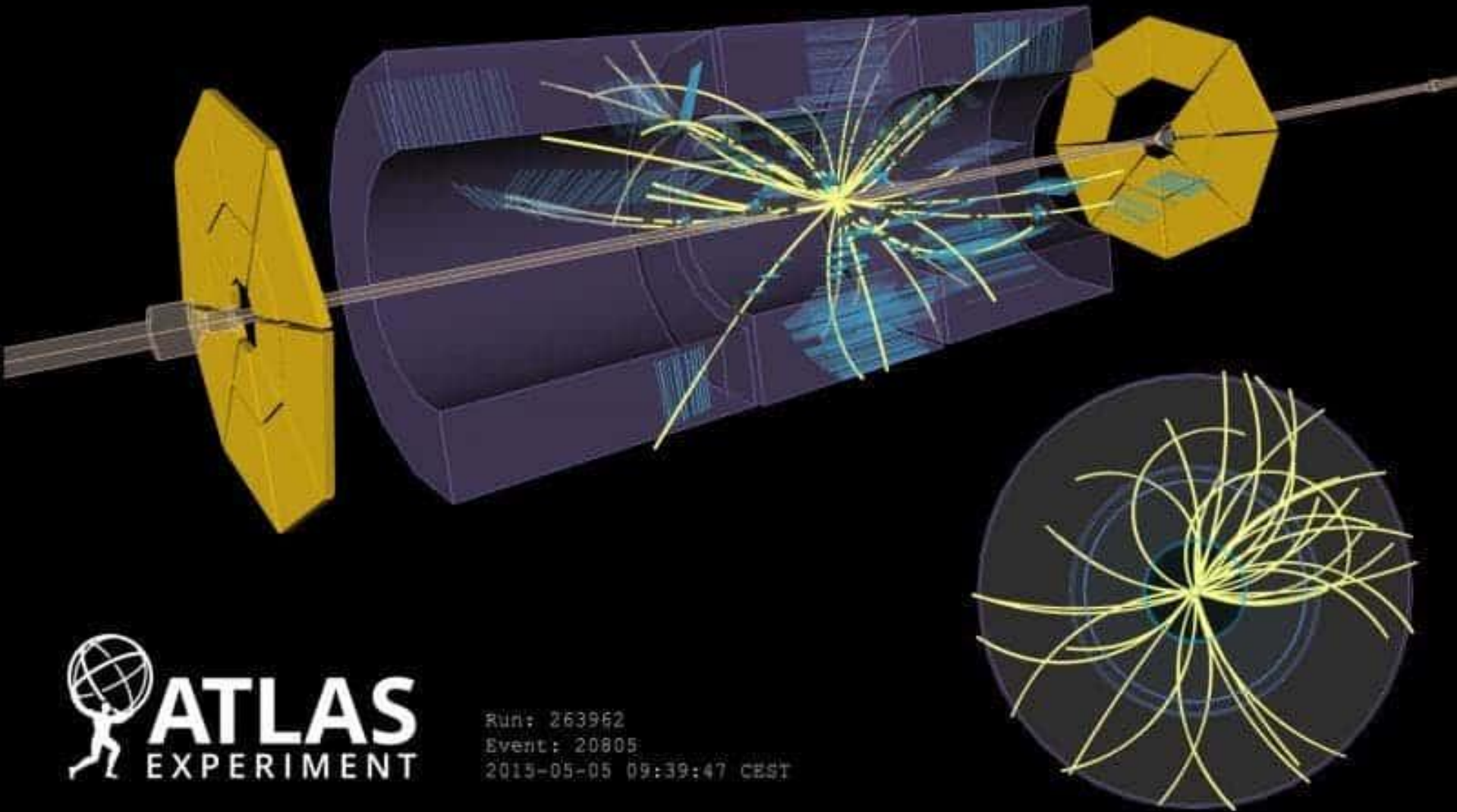
Outline

- Introduction
- Graph Neural Networks @ IceCube
- GNN Tutorial
- Summary

Introduction (1)

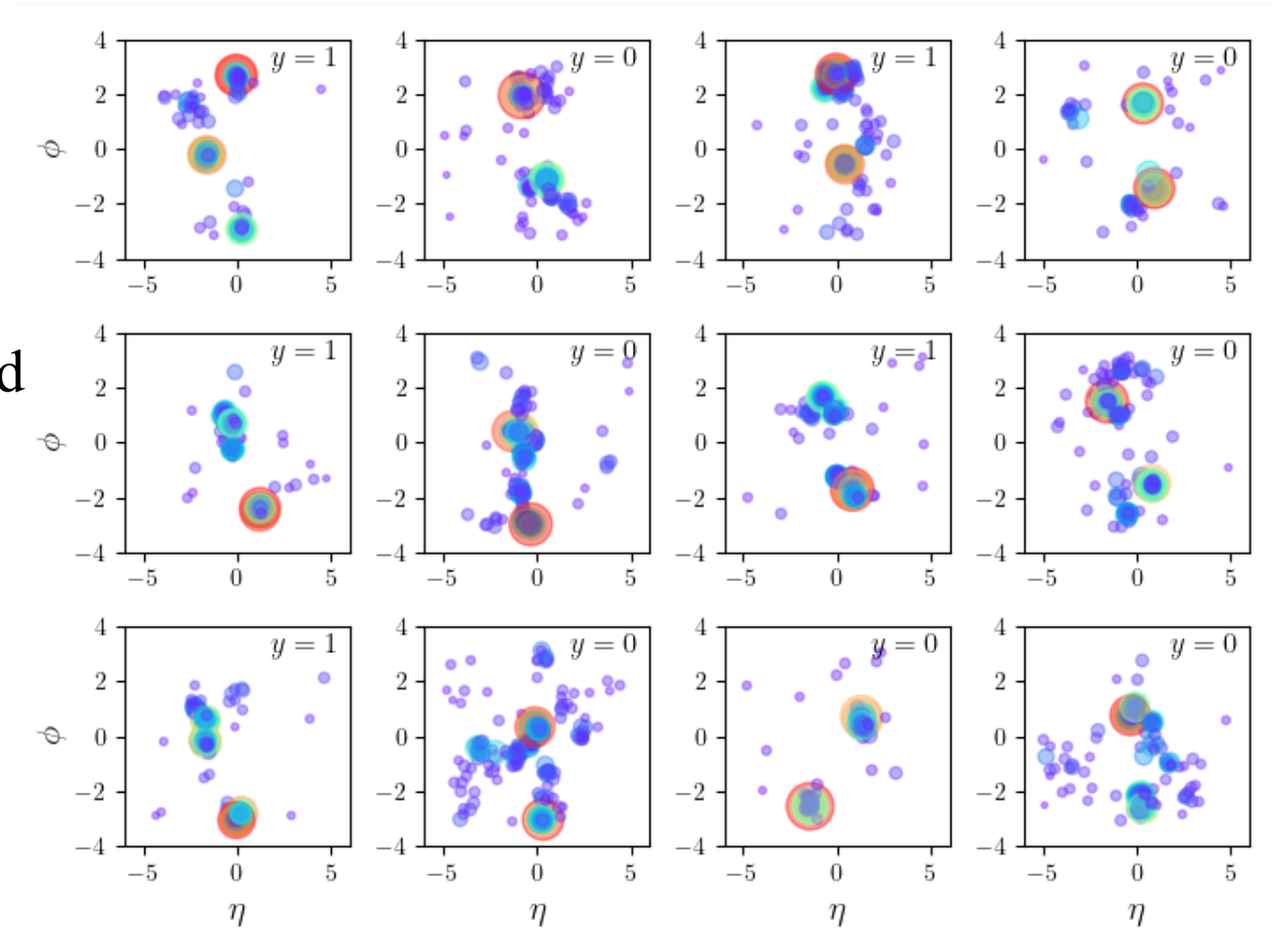
- Any sample $S = \{x_i\}_{i=0}^{n-1}$ of size n within a dataset can be regarded as a cloud of n points in a vector space, that is, as a **point cloud**.
- Standard neural networks expect the size of the point cloud S to be fixed for every sample.
- However, there are datasets in which the size of the point cloud varies from cloud to the next. **Example:** the number of particles created in high-energy particle collision can vary from one collision event to the next.

$$\mathcal{S} = \{ (p_T, \eta, \phi)_i \}_{i=0}^{n-1}$$



Introduction (2)

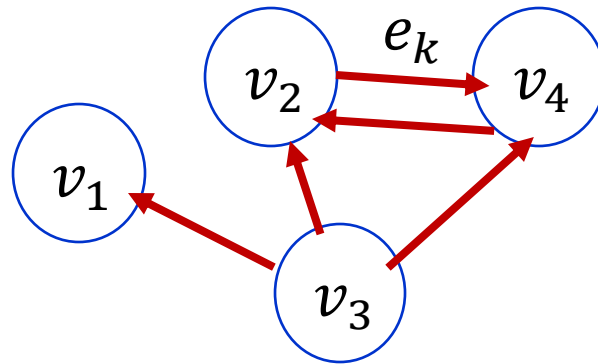
Particle collision events for two classes of interactions, labeled $y = 1$ and $y = 0$.



<https://lhco2020.github.io/homepage/>

Introduction (3)

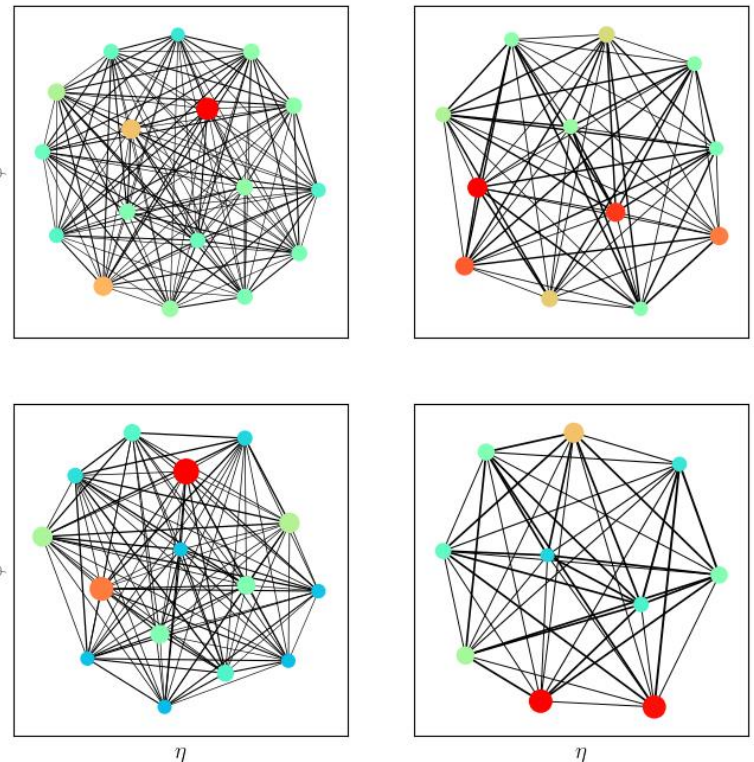
- The data from a single collision event can be modeled as a point cloud with n points in a d -dimensional space.
- One way to represent a point cloud is as a **graph**.



- A graph $G = (V, E)$ comprises vertices (or nodes) and edges, with attributes $V = \{v_i\}$ and $E = \{e_k\}$, respectively.

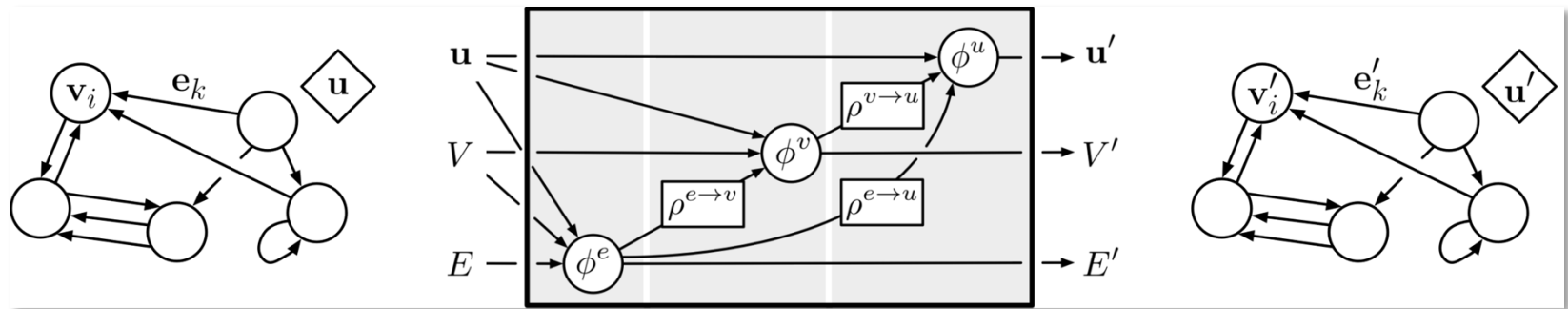
Example: Collision Events as Graphs

- The dots (i.e., nodes) represent particles, with the colors representing different transverse momenta (momentum in the $x - y$ plane).
- The lines (i.e., edges) represent the relationships between the particles, with the thickness of the lines representing the strength of the relationships.



Introduction (4)

A graph neural network is a function that operates on data represented as graphs.

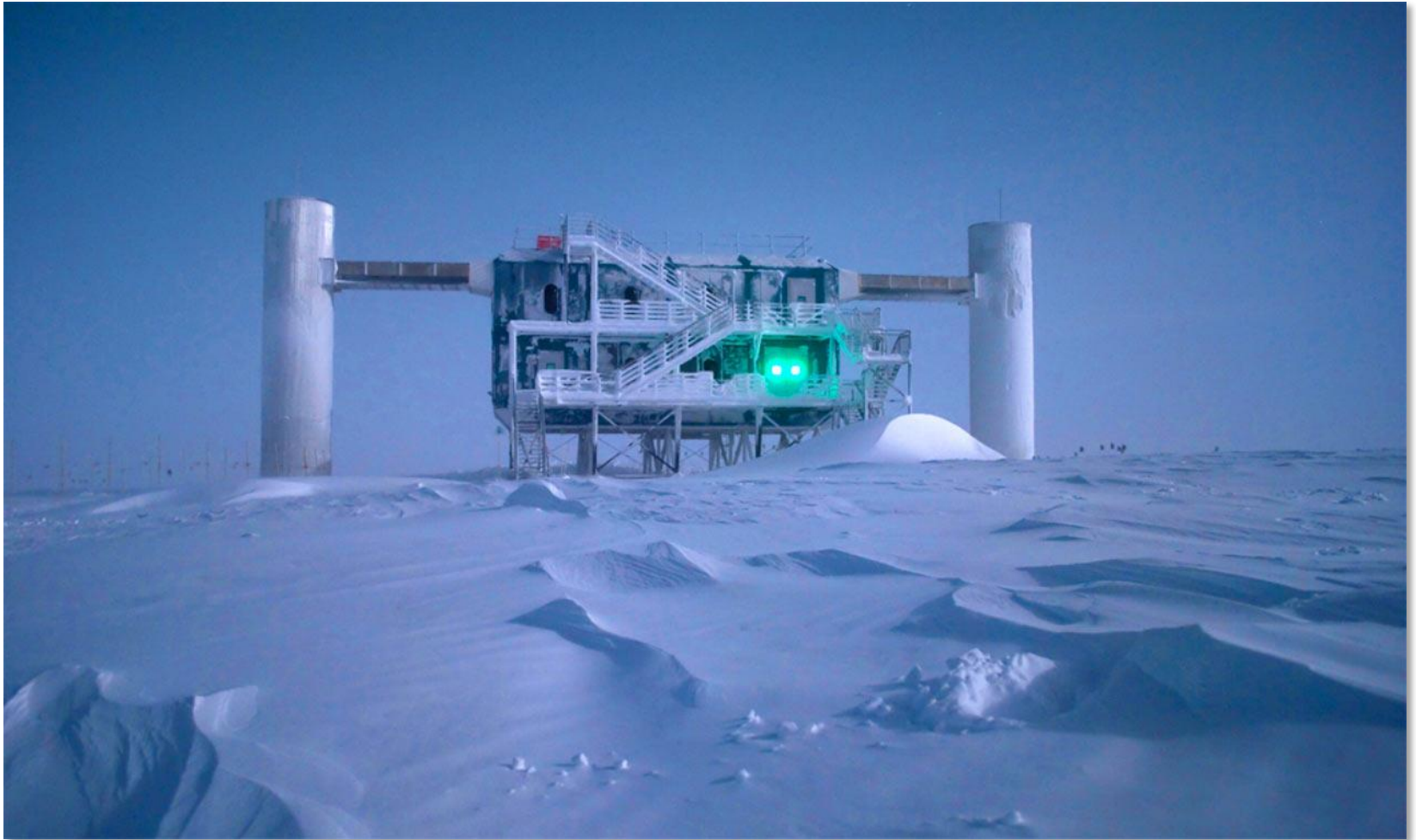


A graph enters a **graph processor** on the left and exits the processor on the right with altered graph attributes.

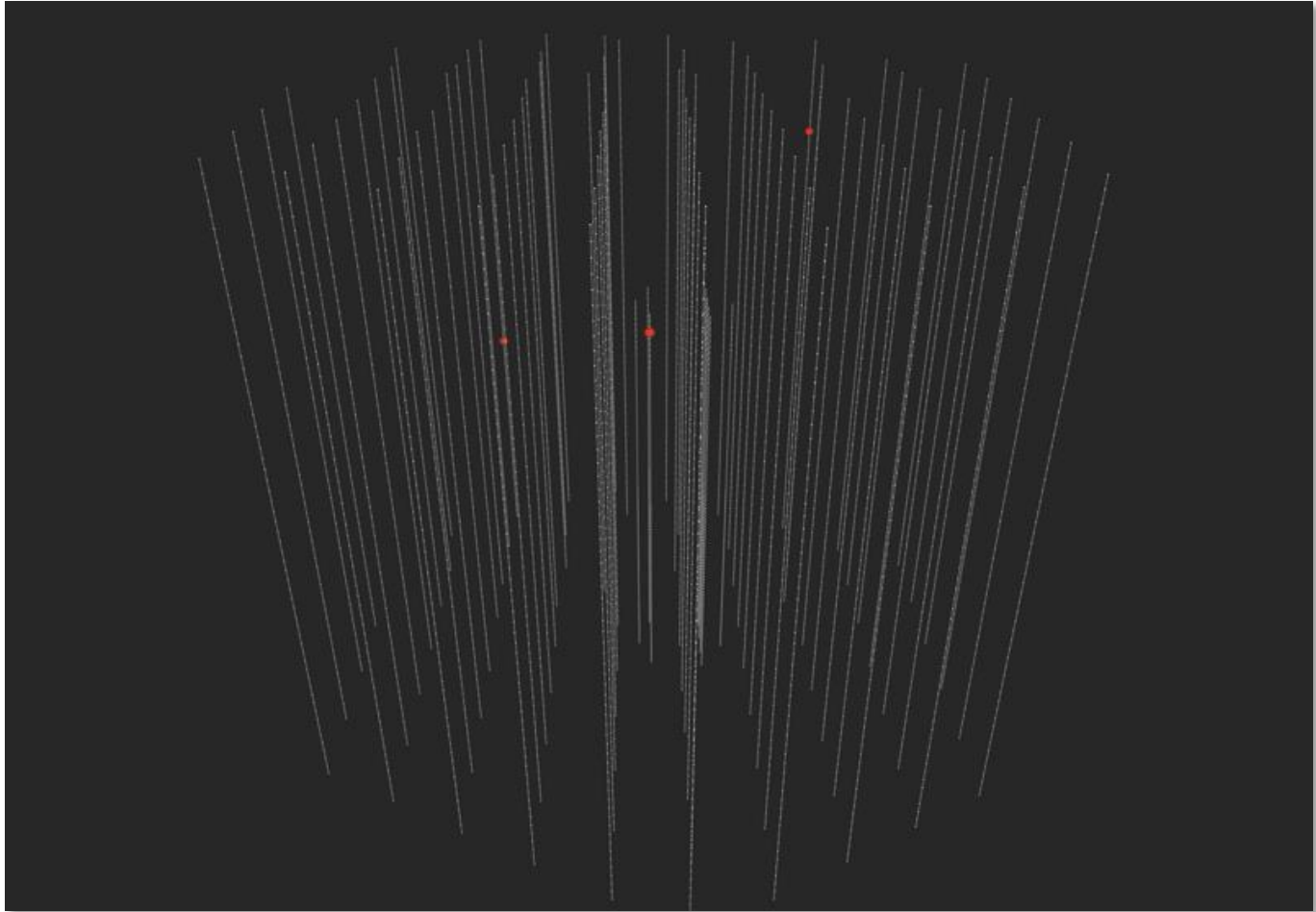
<https://arxiv.org/abs/1806.01261>

GNNS @ IceCube

GNNS @ IceCube (1)

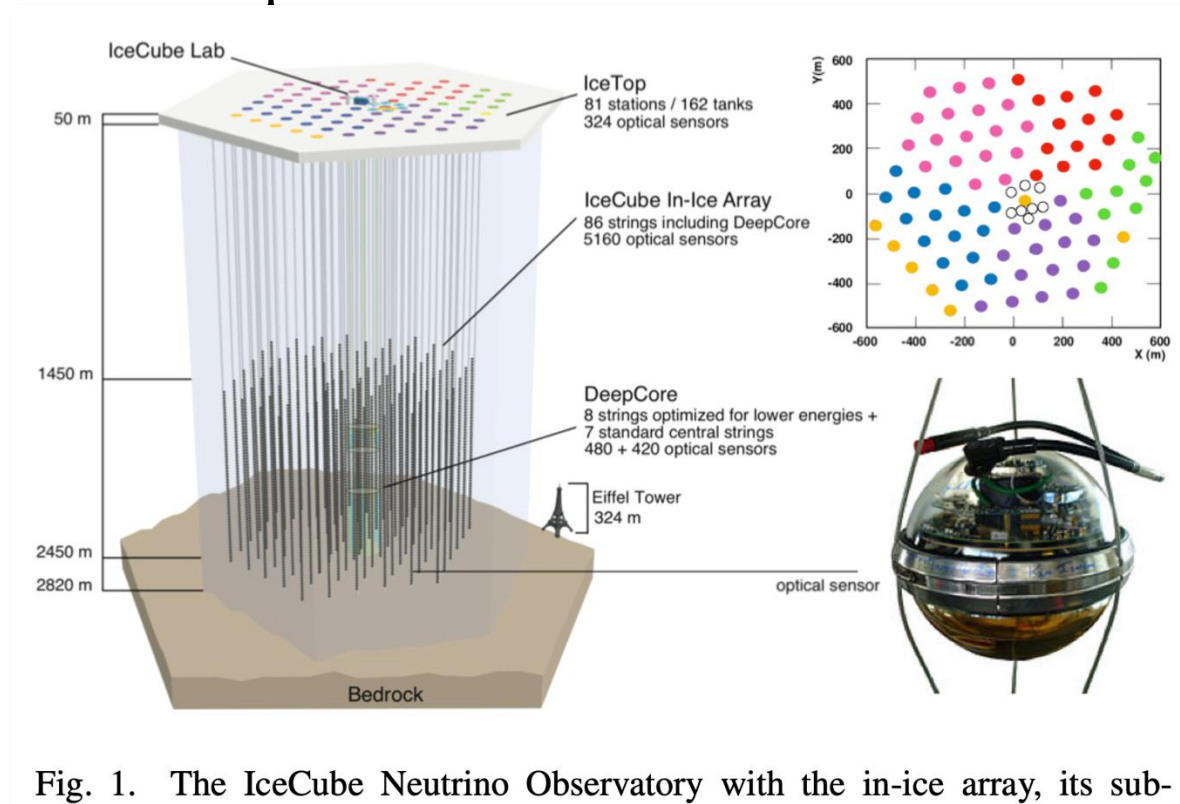


GNNS @ IceCube (2)



GNNS @ IceCube: Data (1)

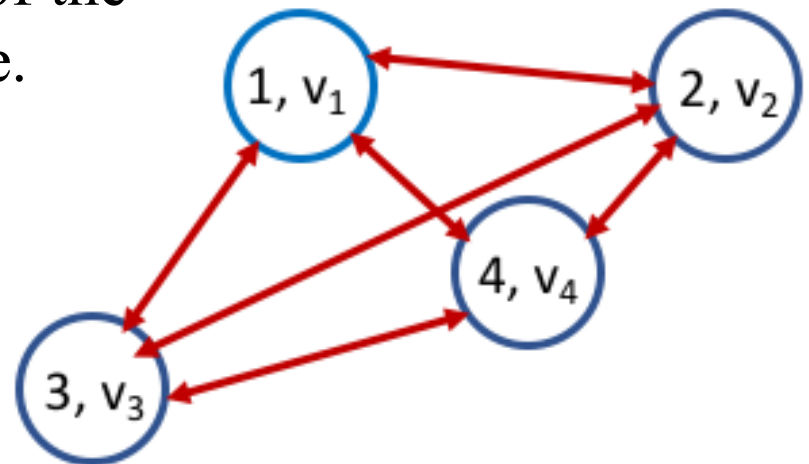
The data collected by the IceCube detector consists of signals from photomultipliers*.



*N. Choma et al. IceCube collaboration, Graph Neural Networks for IceCube Signal Classification, arXiv:1809.06166v1

GNNS @ IceCube: Data (2)

- IceCube data are the signals from n Digital Optical Modules (DOMs). The signals are modeled as the nodes of a graph.
- Each node is associated with a d -dimensional (row-wise) vector of attributes $\mathbf{v}_i = (x_1, \dots, x_d)_i$, three of which are the spatial coordinates (x, y, z) of the associated digital optical module.



GNNS @ IceCube: Data (3)

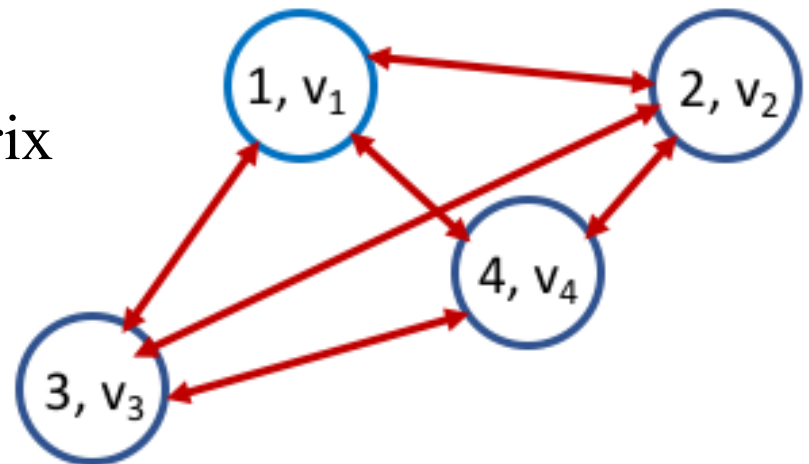
For an event with n signals, an $n \times n$ **adjacency matrix**, $A \equiv E$, is constructed using the spatial data only,

$$(A)_{ij} = \text{softmax}(d_{ij}, \text{dim} = 1),$$

where

$$d_{ij} = \exp\left(-\alpha \|x_i - x_j\|^2\right), \quad \sum_{j=0}^{n-1} d_{ij} = 1$$

and α is a free parameter. The matrix A models the edges of the graph.

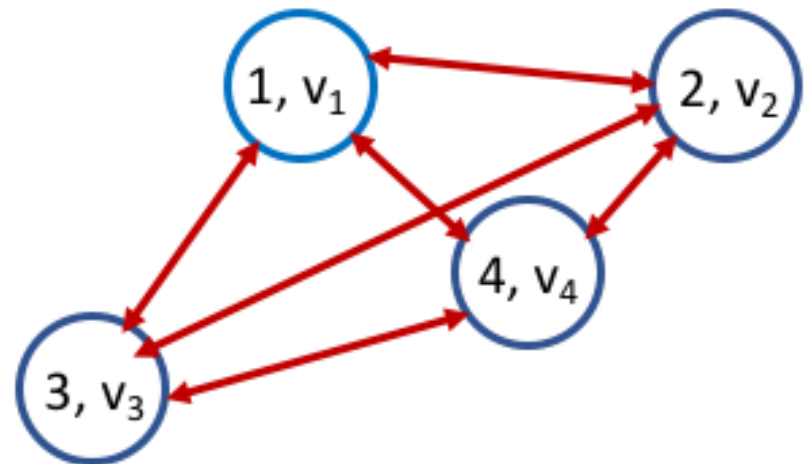


GNNS @ IceCube: Data (4)

The vectors $\mathbf{v}_i$ are concatenated vertically into an $n \times d$ matrix: $V = [\mathbf{v}_1; \cdots; \mathbf{v}_n]$

$$V = \begin{pmatrix} x_1 & y_1 & z_1 & \cdots \\ x_2 & y_2 & z_2 & \cdots \\ \vdots & \vdots & & \vdots \\ x_n & y_n & z_n & \cdots \end{pmatrix}$$

(Note: horizontal concatenation is denoted by $V = [\mathbf{v}_1, \cdots, \mathbf{v}_n]$.)

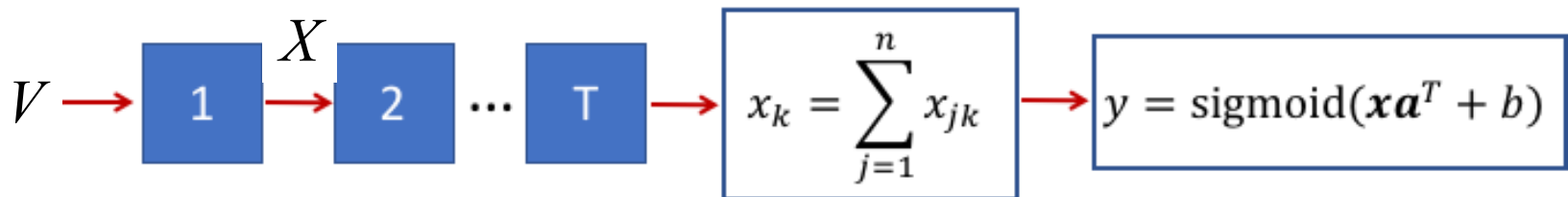


GNNS @ IceCube: GNN (1)

- The $n \times d$ matrix, V , passes through a sequence of T graph processors, each with its own parameters.
- A graph processor is parameterized by a $2d \times d/2$ matrix of parameters, w , and a single scalar b and computes:

$$Y = [AV, V]w + b$$

- The Y operation is an example of a **graph convolution (GNNConv)** and I is an $n \times d/2$ matrix of ones.

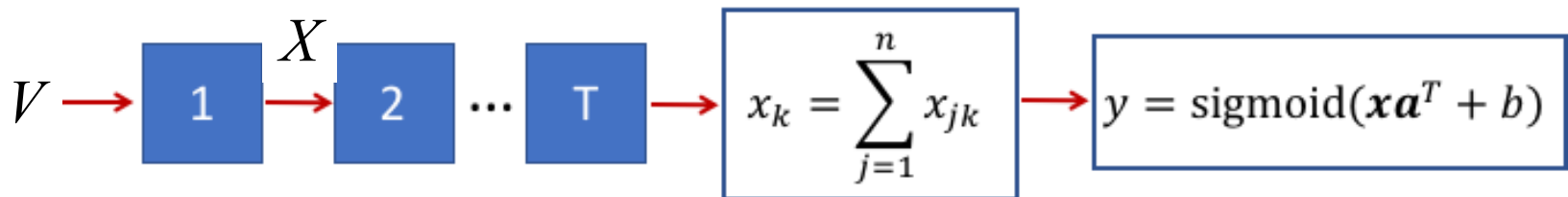


GNNS @ IceCube: GNN (2)

- In IceCube, the vertices, V , of the output graph $G = (V, A)$ are constructed through *horizontal* concatenation as follows:

$$X = [\text{ReLU}(Y), Y]$$

with, as usual, the ReLU function applied *element-wise*.



GNNS @ IceCube: GNNConv (1)

$$Y = [AV, V]w + b,$$
$$X = [\text{ReLU}(Y), Y]$$

Implementation

A = edges(V)

AV = torch.matmul(A, V)

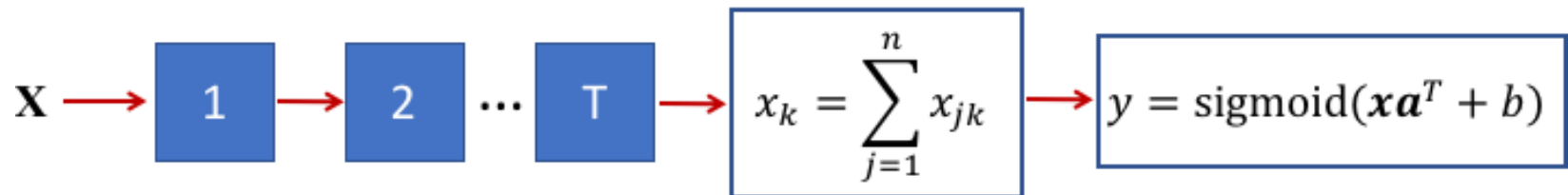
AVV = torch.cat([AV, V], dim=1)

Y = torch.matmul(AVV, w) + b

X = torch.cat([F.relu(Y), Y], dim=1)

GNNS @ IceCube: GNN (3)

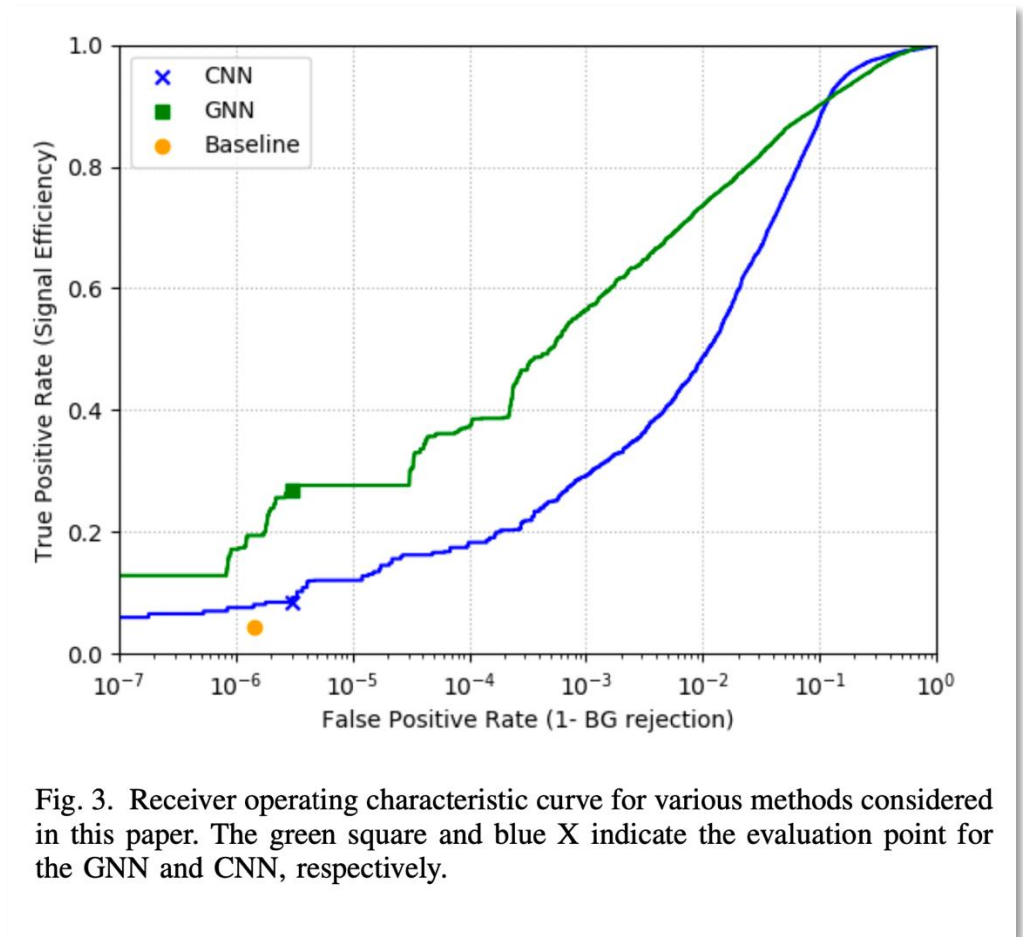
- At the end, $\mathbf{X}$ is mapped to a d -dimensional vector $\mathbf{x}$ using a *permutation invariant map* with respect to the nodes (vertices) and *invariant* with respect to the number of nodes.
- Finally, the vector $\mathbf{x}$ is mapped to the scalar output $0 \leq y \leq 1$ using a *sigmoid*.



GNNS @ IceCube: GNN Results

The GNN is **6.3** times more efficient than the IceCube physics baseline analysis at a signal to noise ratio that is **3** times better.

It also outperforms a 3D CNN.



GNN TUTORIAL

GNN Tutorial: Goal

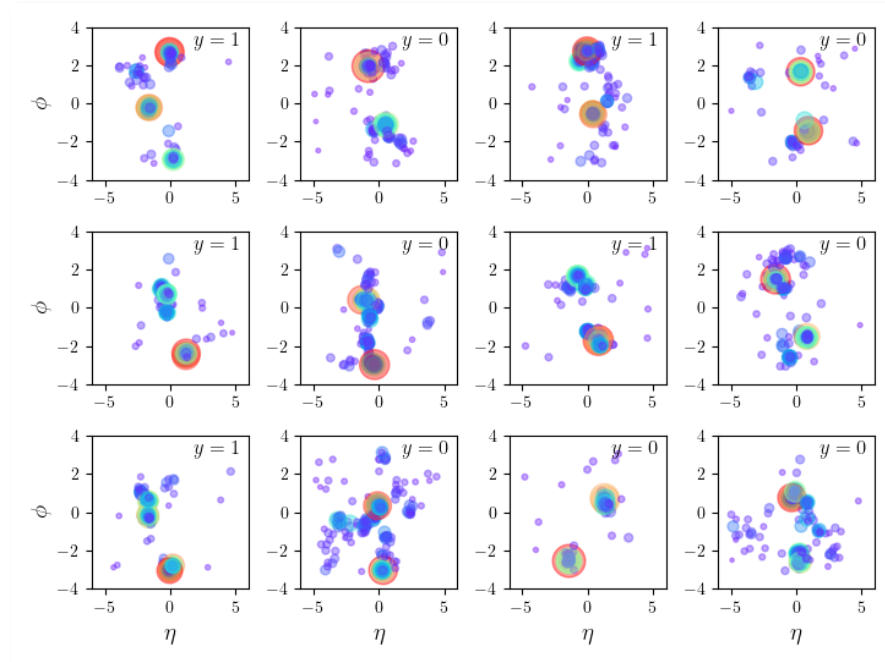
In this tutorial, we distinguish two classes of particle collision events: signal ($y = 1$) or background ($y = 0$), where each event is an array of particles with attributes (p_T, η, ϕ) , the transverse momentum, i.e., the momentum in the $x - y$ plane, and the particle direction.

We use a simpler graph convolution than IceCube's:

$$X = \sigma(XW_1^T + B_1) \quad \text{embed}$$

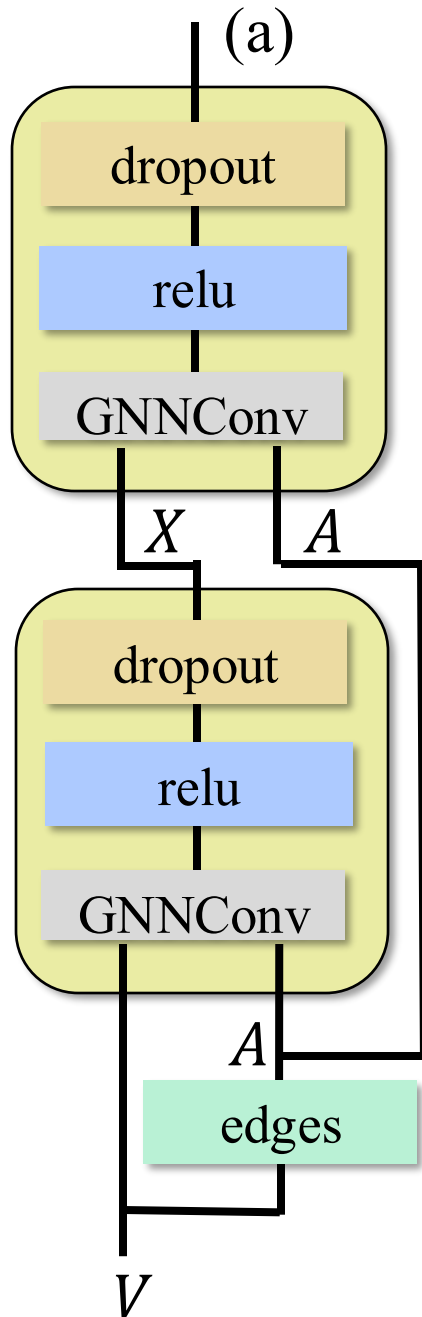
$$X' = AX$$

$$X'' = X'W_2^T + B_2$$



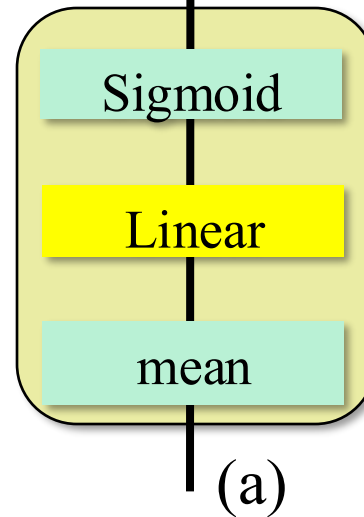
<https://lhco2020.github.io/homepage/>

GNN: Model

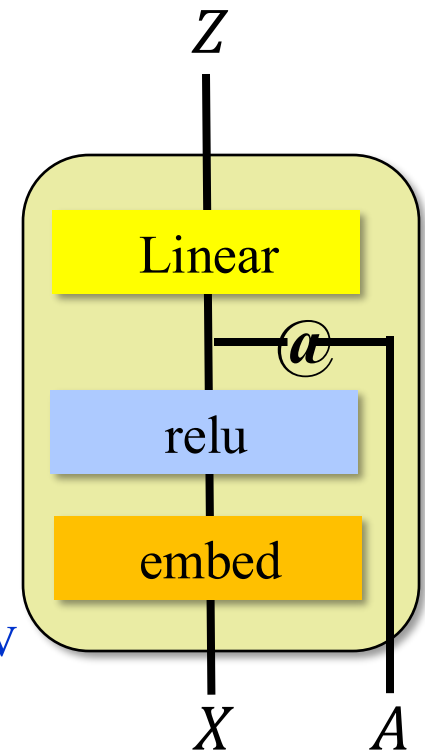


$$p(y = 1|G),$$

$$G = (V, A)$$



GNNConv



```
class GNNConv(nn.Module):

    def __init__(self, in_features, out_features):

        # initialize base class
        super().__init__()

        # embed the "n" input vertices in a larger vector space
        # of dimension h = out_features

        self.embed = nn.Linear(in_features, out_features)

        self.linear = nn.Linear(out_features, out_features)
```

Implementation: graph convolution (this slide and the next).


```
def forward(self, X, A):
```

```
    # embed vertices in a larger vector space
```

```
    # compute  $(A * X) * W1 + B1$ 
```

```
    X = self.embed(X)
```

```
    X = F.relu(X)
```

```
    # perform graph convolution: for each vertex, compute a
```

```
    # weighted sum of the features of all other vertices,
```

```
    # weighting the nearest neighbors more heavily.
```

```
    #  $AX = A * X$ 
```

```
    AX = torch.matmul(A, X)
```

```
    # compute  $(A * X) * W2 + B2$ 
```

```
    X = self.linear(AX)
```

```
    # return modified vertices
```

```
    return X
```

```
class GNN(mlp.Model):  
  
    def __init__(self, n_dim=3, n_embed=40, dropout=0.5):  
  
        # initialize base class  
        super().__init__()   
  
        self.edges    = Edges()  
        self.gconv0   = GNNConv(n_dim,  n_embed)  
        self.gconv1   = GNNConv(n_embed, n_embed)  
        self.linear    = nn.Linear(n_embed, 1)  
        self.dropout  = nn.Dropout(dropout)
```

Implementation: GNN (this slide and the next).

```

def forward(self, V):
    # construct n x n adjacency matrix which together
    # with the nodes (vertices) V form the graph  $G = (V, A)$ 
    A = self.edges(V).

    # pass graph  $G = (V, A)$  through a couple
    # graph convolutions
    X = self.gconv0(V, A); X = F.relu(X); X = self.dropout(X)
    X = self.gconv1(X, A); X = F.relu(X); X = self.dropout(X)

    # apply a permutation and node (vertex) count-invariant
    # function to the nodes (vertices)
    x = torch.mean(X, dim=1)

    # complete with a simple MLP
    x = self.linear(x)
    x = F.sigmoid(x)
    return x

```

Summary

- Data instances in which the number of data points varies can be modeled with graphs $G = (V, E)$ using graph neural networks (GNN).
- We looked at a specific application of GNNs by IceCube, which, for this experiment, achieved state-of-the-art classification results.